

โครงงาน

โนวา เอส-เคฟ

(Nova Escape)

จัดทำโดย

6804062612323 นายธนภัจน์ ขนาบแก้ว

Section 1

อาจารย์ผู้สอน

ผู้ช่วยศาสตราจารย์ ดร. สถิตย์ ประสมพันธ์

โครงงานนี้เป็นส่วนหนึ่งของวิชา

040613204 Object-Oriented Programming

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาคเรียนที่ 1 ปีการศึกษา 2569

Source Code: https://github.com/6804062612323-Thanapat/NovaEscape

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของโปรเจ็ค

โครงงานนี้จัดทำขึ้นเพื่อนำความรู้จากวิชา Object-Oriented Programming มาสร้างเป็นเกมที่เล่นได้จริงด้วยภาษา Java ผู้จัดทำเลือกทำเกมเพราะเกมมีวัตถุจำนวนมากที่ต้องทำงานร่วมกัน เช่น ยาน ศัตรู กระสุน และไอเทม จึงเห็นประโยชน์ของการออกแบบเป็นคลาสได้ชัดเจน

แนวคิดตั้งต้นตาม Proposal คือเกม 2D ที่ผู้เล่นควบคุมยานอวกาศหลบเลเซอร์จากยานของมนุษย์ต่างดาว เล่นเป็นตา และยากขึ้นทุกตา ต่อมาผู้จัดทำขยายเกมด้วยโหมด ASSAULT ซึ่งเป็นเกมยิงแนว Metal Slug เพื่อให้มีศัตรูและด่านหลายแบบ และได้ใช้ Inheritance, Polymorphism และ Abstract Class อย่างเต็มที่

1.2 ประเภทของโครงการ

เกม (Game) แบบ 2D ผู้เล่นคนเดียว ทำงานบนคอมพิวเตอร์เป็น Java Desktop Application

1.3 ประโยชน์

ประโยชน์ต่อผู้เล่น (ตาม Proposal)

1. ฝึกไหวพริบในการมอง (reflex)
2. ฝึกการคิดวิเคราะห์และการตัดสินใจ
3. เพื่อความสนุกสนาน

ประโยชน์ต่อผู้จัดทำ

1. ได้ใช้หลักการ OOP ทั้ง 6 หัวข้อในโปรแกรมขนาดใหญ่ (ประมาณ 7,900 บรรทัด)
2. ได้ฝึกเขียน GUI และจัดการ Event ด้วย Java Swing
3. ได้ฝึกออกแบบอัลกอริทึมของเกม เช่น game loop การตรวจจับการชน และระบบความยาก

1.4 ขอบเขตของโครงการ

เกมที่ส่งจริงทำครบตามขอบเขตใน Proposal และเพิ่มโหมด ASSAULT จากแผนเดิม ตารางนี้สรุปขอบเขตของเกมฉบับสมบูรณ์

หัวข้อ	ขอบเขต
ภาษาและไลบรารี	Java 8 ขึ้นไป ใช้ Swing, AWT และ Java Sound ไม่มีไลบรารีภายนอก
รูปแบบโปรแกรม	Desktop Application หน้าต่างขนาด 960 × 540 พิกเซล
ผู้เล่น	1 คน ควบคุมด้วยคีย์บอร์ดและเมาส์
โหมด CLASSIC	หลบเลเซอร์ตาละ 20 วินาที เลือกไอเทมระหว่างตา ความยากเพิ่มทุกตา
โหมด ASSAULT	ยิงต่อสู้ 3 ด่าน มีมินิบอสและบอสประจำด่าน มีรถถัง Nova Slug ด่านละ 1 คัน
ภาพและเสียง	วาดและสังเคราะห์ด้วยโค้ดทั้งหมด ไม่มีไฟล์ภาพหรือไฟล์เสียงในโปรเจกต์
การบันทึกข้อมูล	เก็บคะแนนสูงสุดของแต่ละโหมดเป็นไฟล์ `.dat`
สิ่งที่ส่ง	ไฟล์ JAR, Source Code, เอกสารรายงาน และ Source Code บน GitHub

สิ่งที่อยู่นอกขอบเขต: การเล่นหลายคน การเล่นผ่านเครือข่าย และการรันบนเว็บหรือมือถือ

เอกสารแนบ Proposal

หน้าต่อไปนี้คือ Proposal ฉบับที่เสนอไว้ก่อนเริ่มพัฒนา

นายธนภัจน์ ขนาบแก้ว 6804062612323 sec 1

โนวา เอส-เคฟ
(NOVA ESCAPE)

- รายละเอียดเกมส์

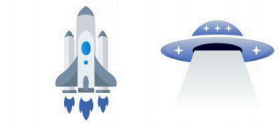
เกม 2D ที่ผู้เล่นในรับบทควบคุมยานอวกาศเพื่อหลบสิ่งกีดขวางต่างๆ ที่ปล่อยมาจากยานบัญชาของมนุษย์ต่างดาว โดยเกมจะเล่นเป็นคดโดยเมื่อผ่านตามหนึ่งไปจะสามารถเลือกเล่นต่อหรือหยุดเกมได้ หากเล่นต่อจะมีไอเทมให้เลือกเพื่อทำให้เล่นง่ายขึ้น เล่นซ้ำรู้จะมีความยากมากขึ้น โดยผู้เล่นต้องพยายามเอาชีวิตรอดให้นานที่สุดและทำคะแนนให้มากที่สุด

- วิธีการเล่น

-ควบคุมยานด้วย WASD เพื่อควบคุมทิศทาง (W เพื่อไปข้างหน้า, S ไปข้างหลัง, A ไปทางซ้าย, S ไปทางขวา)
-กดใช้ไอเทมเสริมด้วยปุ่ม Spacebar
-เมาส์สำหรับการใช้กดปุ่มต่างๆ เช่นกดปุ่ม Start เริ่มเกม

- Storyboard

-ตัวละคร



ยานอวกาศของผู้เล่น ยานของมนุษย์ต่างดาว

-ฉาก

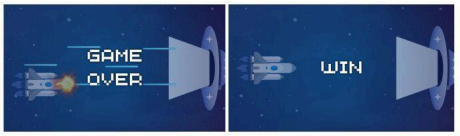
หน้าเริ่มเกม



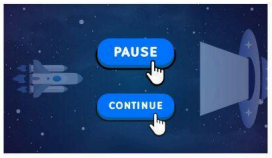
เกมเพย์ขณะเล่นเกม



เมื่อชนยานชนกับเลเซอร์ (Game Over) / เมื่อชนยานหลบเลเซอร์จนครบเวลา (Win)



เมื่อจบเกมจะมีให้เลือก หยุดเกมหรือเล่นเกมต่อเพื่อสะสมแต้มคะแนนเพิ่ม



ถ้าเลือกเล่นเกมต่อจะมีไอเทมให้เลือก
-หัวใจ เมื่อผู้เล่นถูกยิงจะสามารถชุบชีวิตได้ 1 ครั้ง
-ไอ่ เมื่อกดใช้ไอ่จะทำให้ผู้เล่นเป็นอมตะ 5 วินาที (ชนเลเซอร์แล้วไม่ตาย)
-โจมตี เมื่อกดใช้ไอเทมโจมตี จะทำให้ยานอวกาศของศัตรูหยุดยิงเป็นเวลา 5 วินาที



- ประโยชน์

1.ฝึกไหวพริบในการมอง (reflex)
2.ฝึกการคิดวิเคราะห์ การตัดสินใจ
3.เพื่อความสนุกสนาน

- ตารางแสดงการทำงาน

ลำดับ	รายการ	1-7 ก.ย.	8-14 ก.ย.	15-24 ก.ย.	25-30 ก.ย.
1	หาอุปกรณ์ของครบและทำการฟิกต่างๆ				
2	ศึกษาและหาข้อมูลที่เกี่ยวข้อง				
3	ลงมือเขียนโปรแกรม				
4	จัดทำเอกสาร				
5	ตรวจสอบและแก้ไขข้อผิดพลาด				

บทที่ 2

ส่วนการพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

เนื้อเรื่องย่อ

ผู้เล่นรับบทนักบินยานอวกาศที่ต้องหนีจากยานปริศนาของมนุษย์ต่างดาว ในโหมด CLASSIC ยานไม่มีอาวุธ จึงต้องหลบเลเซอร์ให้รอดนานที่สุด ในโหมด ASSAULT ผู้เล่นติดอาวุธบุกฝ่า 3 เขต ได้แก่ Nova Sector, Frozen Rings และ Hive Core พร้อมช่วยนักบินอวกาศที่ลอยอยู่และทำลายบอสของแต่ละเขต

ปุ่มควบคุม

ปุ่ม	โหมด CLASSIC	โหมด ASSAULT
W A S D หรือลูกศร	บังคับยาน	บังคับยาน
Spacebar	ใช้ไอเทมชิ้นแรกในคิว	ใช้ไอเทมชิ้นแรกในคิว (โล่ / EMP)
1 2 3 หรือคลิก	เลือกไอเทมหลังผ่านตา	ไม่ใช้
J หรือคลิกซ้ายค้าง	ไม่ใช้	ยิง
K หรือคลิกขวา	ไม่ใช้	Nova Bomb หรือปืนใหญ่เมื่ออยู่ในรถถัง
เมาส์	กดปุ่มบนหน้าจอ	กดปุ่มบนหน้าจอ และเล็งป้อมปืนรถถัง
P หรือ ESC	ไม่ใช้	หยุดเกมชั่วคราว
M	เปิด/ปิดเสียง	เปิด/ปิดเสียง

โหมด CLASSIC

1. หลบเลเซอร์ให้ครบ 20 วินาทีจะได้ WIN แต่ถ้าชนเลเซอร์จะ GAME OVER
2. เมื่อชนะ เลือก PAUSE เพื่อหยุดและสรุปคะแนน หรือ CONTINUE เพื่อเล่นต่อ
3. ถ้าเล่นต่อ เลือกไอเทม 1 อย่าง แล้วตาถัดไปศัตรูจะยิงเร็วขึ้น ถี่ขึ้น และเล็งแม่นขึ้น

ไอเทมมี 3 แบบ คือ **หัวใจ** ชุบชีวิตอัตโนมัติ 1 ครั้ง, **โล่** อมตะ 5 วินาที และ **โจมตี** ทำให้ยานศัตรูหยุดยิง 5 วินาที คะแนนได้จาก 10 × เลขตาต่อวินาทีที่รอด บวกโบนัสผ่านตา 100 × เลขตา

โหมด ASSAULT

เป้าหมายคือบินไปจนสุดแมพแล้วชนะบอส ผู้เล่นเริ่มด้วย HP 5 ช่อง ชีวิตสำรอง 2 ชีวิต และ Nova Bomb 2 ลูก (ถือได้สูงสุด 6 ลูก) จบด่านแล้วกด NEXT STAGE เพื่อเล่นต่อโดยยกคะแนน ชีวิต และระเบิดไปด่านถัดไป

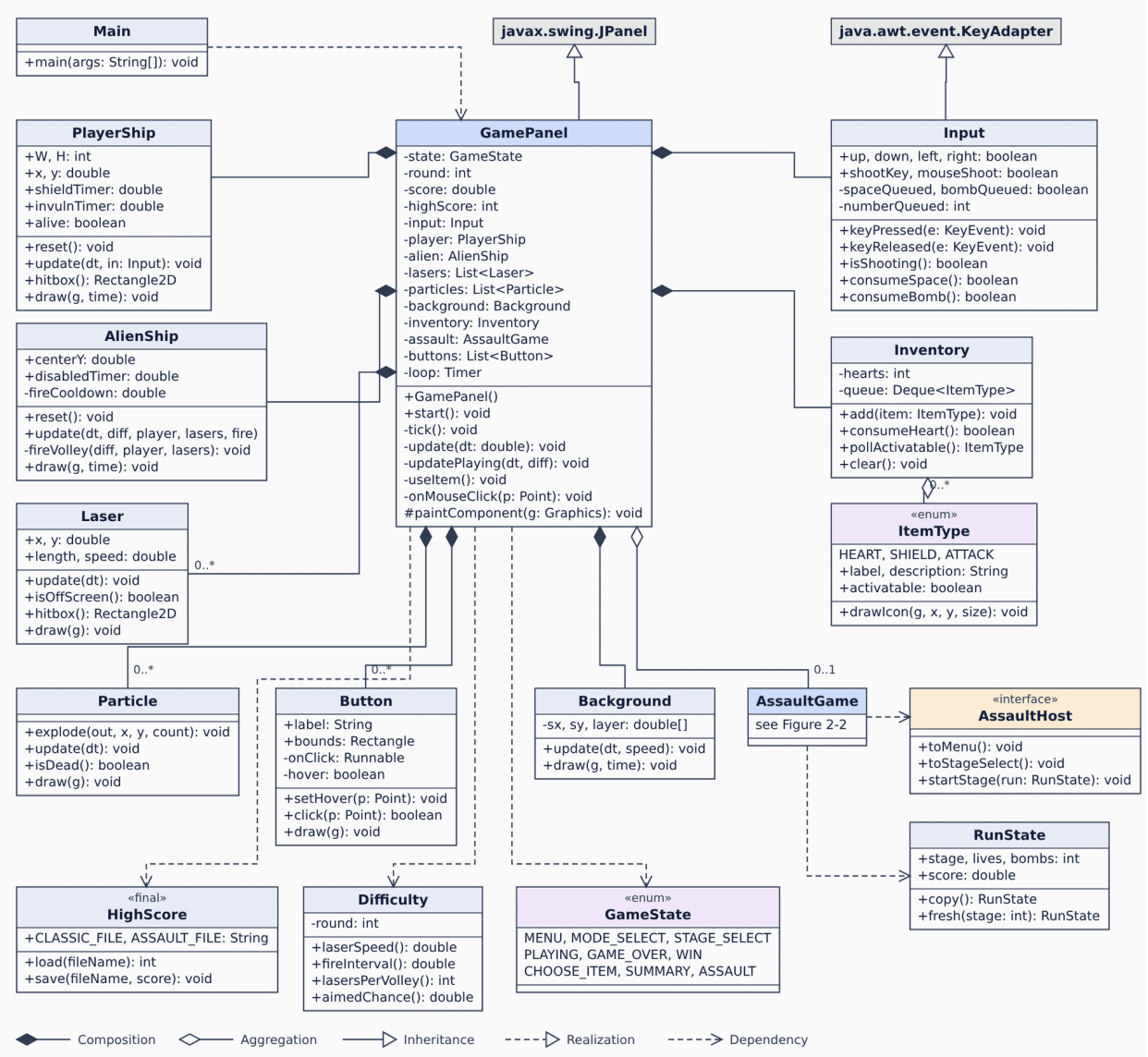
ด่าน	เขตย่อย	กลไกพิเศษ	มินิบอส	บอส
1 Nova Sector	3 เขต	อุกกาบาตและฐานต่างดาว	Carrier	Mothership (HP 420)
2 Frozen Rings	4 เขต	พายุหิมะผลักยานถอยหลัง	Frost Warden	Cryo Serpent (HP 360)
3 Hive Core	6 เขต	ผนังอุโมงค์บีบเข้ามา ชนแล้วเจ็บ	Brood Mother	Hive Queen (ตา 3 × 100 + หัวใจ 220)

ระบบอื่น ๆ

- **รถถัง Nova Slug:** บินไปชนเพื่อขึ้นรถ ป้อมปืนเล็งตามเมาส์ มีเกราะ 12 ช่อง ใช้ได้ 25 วินาที เก็บกล่องปืนเพิ่มเวลาได้กล่องละ 1 วินาที สะสมสูงสุด 35 วินาที
- **นักบินอวกาศ (HELP!):** บินไปชนเพื่อช่วย ได้ทันที 500 คะแนนและไอเทม 1 ชิ้น จบด่านได้โบนัสอีกคนละ 1,000 คะแนน
- **ปืน 5 แบบ:** PISTOL, HEAVY MG, SPREAD, LASER และ ROCKET เก็บจากกล่องอาวุธเพื่อเปลี่ยนปืน PISTOL กระสุนไม่จำกัด ส่วนปืนอื่นมีกระสุนจำกัด

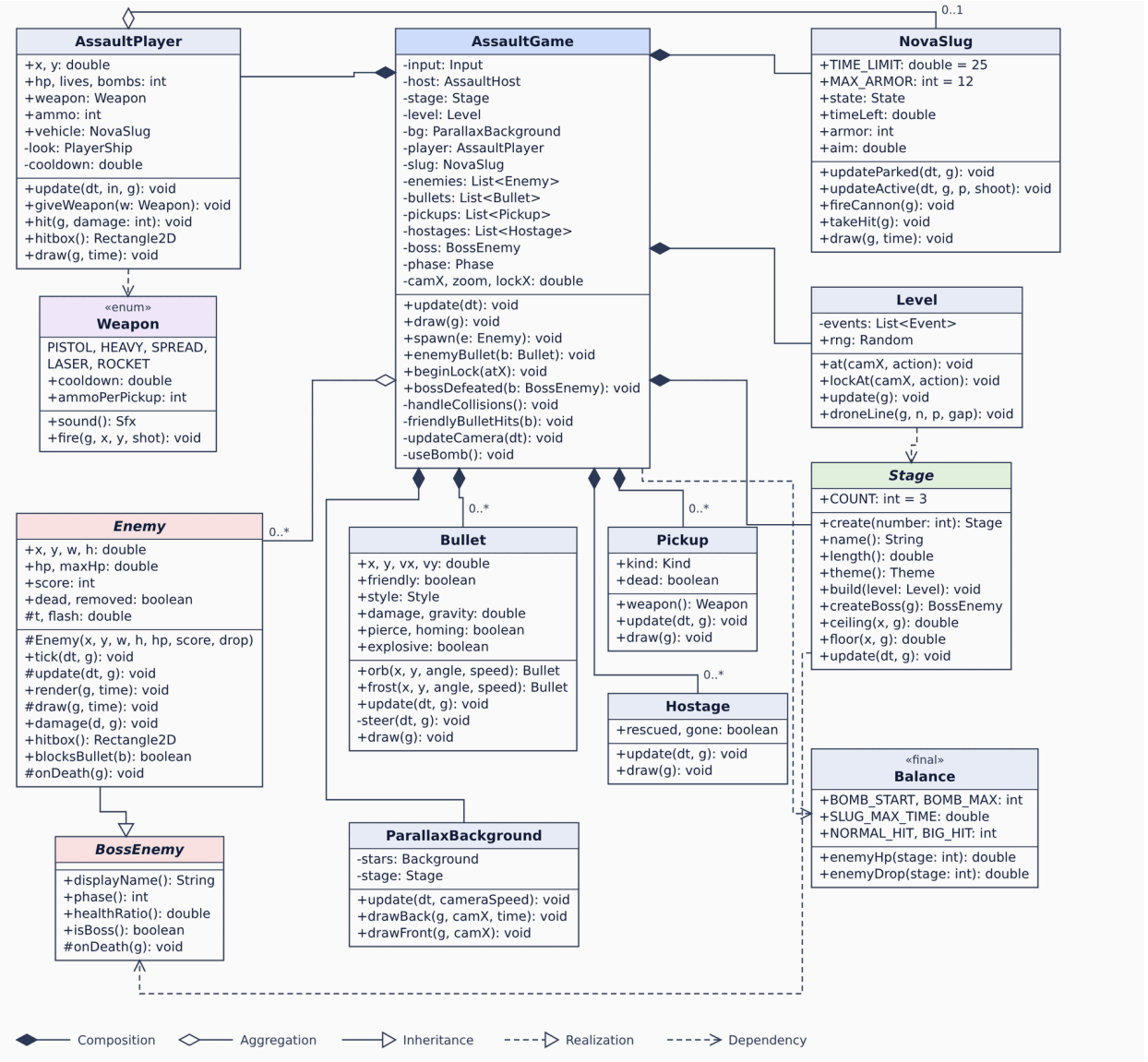
2.2 คลาสไดอะแกรมพร้อมคำอธิบาย

โปรแกรมมีซอร์สโค้ดที่ใช้งาน 57 ไฟล์ แบ่งเป็น 8 แพ็กเกจ รวมประมาณ 7,900 บรรทัด คลาสไดอะแกรมจึงแยกเป็น 3 รูป โดยแสดงเฉพาะแอตทริบิวต์และเมธอดหลักเพื่อให้อ่านง่าย สัญลักษณ์หน้าสมาชิก + คือ public, - คือ private และ # คือ protected



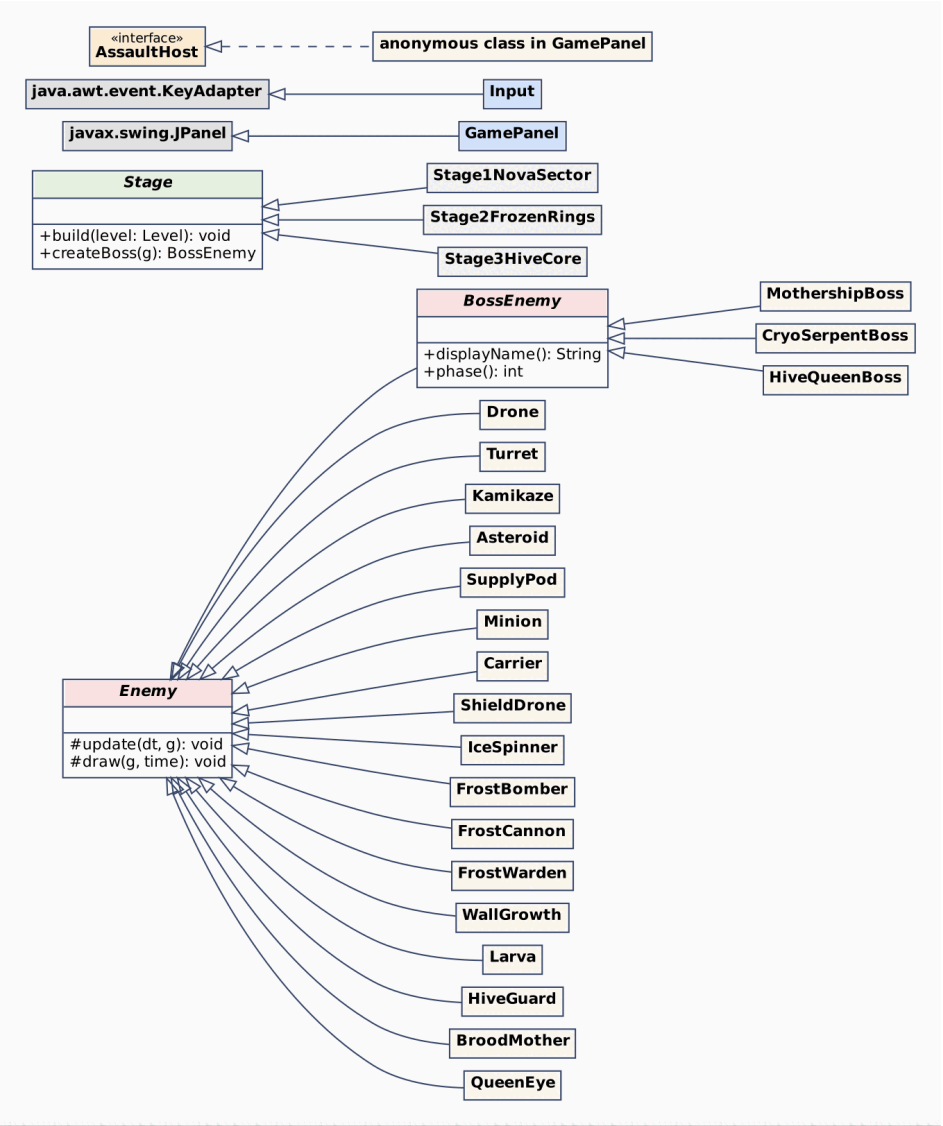
รูปที่ 2-1 คลาสไดอะแกรมโครงสร้างหลักและโหมด CLASSIC

`GamePanel` เป็นศูนย์กลางของโปรแกรม โดยสืบทอดจาก `JPanel` และเป็นเจ้าของวัตถุหลักของโหมด CLASSIC ส่วน `AssaultGame` จะถูกสร้างเมื่อผู้เล่นเข้าโหมด ASSAULT เท่านั้น จึงมี multiplicity เป็น 0..1



รูปที่ 2-2 คลาสไดอะแกรมโหมด ASSAULT

`AssaultGame` เป็นตัวควบคุมโหมดนี้ทั้งหมด ทั้งการอัปเดต การชน กล้อง และการวาด โดยรู้จักศัตรูผ่านคลาสแม่ `Enemy` และรู้จักด่านผ่านคลาสแม่ `Stage` เท่านั้น การเพิ่มศัตรูหรือด่านใหม่จึงแทบไม่ต้องแก้ `AssaultGame`



รูปที่ 2-3 ลำดับการสืบทอดของคลาสในโปรแกรม

หน้าที่ของแต่ละแพ็กเกจ

แพ็กเกจ	จำนวนไฟล์	คลาสสำคัญ	หน้าที่
`novaescape`	7	`Main`, `GamePanel`, `Input`, `Difficulty`	จุดเริ่มโปรแกรม game loop เมนู และโหมด CLASSIC
`novaescape.entity`	4	`PlayerShip`, `AlienShip`, `Laser`, `Particle`	วัตถุในโหมด CLASSIC และอนุภาคที่ใช้ร่วมกัน
`novaescape.item`	2	`Inventory`, `ItemType`	กระเป๋าและชนิดของไอเทม
`novaescape.ui`	3	`Button`, `Background`, `Text`	ปุ่ม ฉากหลัง และการวาดข้อความ
`novaescape.audio`	4	`Sound`, `Sfx`, `Music`, `Synth`	สังเคราะห์และมิกซ์เสียง
`novaescape.assault`	12	`AssaultGame`, `AssaultPlayer`, `NovaSlug`, `Bullet`, `Level`	ระบบหลักของโหมด ASSAULT
`novaescape.assault.enemy`	21	`Enemy`, `BossEnemy`, `HiveQueenBoss`	ศัตรูทั่วไป มินิบอส และบอส
`novaescape.assault.stage`	4	`Stage`, `Stage1NovaSector`	ข้อมูล สคริปต์ และกลไกของแต่ละด่าน

2.3 รูปแบบการพัฒนา Application / Applet

โปรแกรมนี้พัฒนาในรูปแบบ **Application** ไม่ใช่ Applet เพราะเริ่มทำงานจากเมธอด `main` และแสดงผลในหน้าต่าง `JFrame` ของตัวเอง ผู้เล่นเปิดได้จากไฟล์ `NovaEscape.jar` โดยไม่ต้องใช้เว็บเบราว์เซอร์

```
public static void main(String[] args) {
    novaescape.audio.Sound.init();   // เตรียมเสียง (ทำงานเบื้องหลัง)
    SwingUtilities.invokeLater(() -> {
        JFrame frame = new JFrame("Nova Escape");
        GamePanel panel = new GamePanel();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setResizable(false);
        frame.add(panel);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
        panel.start();
    });
}
```

`SwingUtilities.invokeLater` ทำให้การสร้าง GUI เกิดบน Event Dispatch Thread ซึ่งเป็นเธรดที่ Swing กำหนดให้ใช้จัดการหน้าจอ

2.4 อธิบายส่วนของโปรแกรมตามหลัก OOP

โค้ดทุกตัวอย่างในรายงานคัดมาจากซอร์สโค้ดจริงของเกม และตัดบางบรรทัดออกเพื่อให้เห็นประเด็นชัดขึ้น

2.4.1 Constructor

Constructor ใช้กำหนดค่าเริ่มต้นของวัตถุทันทีที่ถูกสร้าง เกมนี้ใช้ Constructor ทั้งแบบตั้งค่าธรรมดา แบบเรียกคลาสแม่ด้วย `super` และแบบสร้างวัตถุอื่นมาประกอบกัน

ตัวอย่างที่ 1: Constructor ของคลาสแม่ `Enemy`

```
protected Enemy(double x, double y, double w, double h, double hp, int score, double
dropChance) {
    this.x = x; this.y = y; this.w = w; this.h = h;
    this.hp = this.maxHp = hp;
    this.score = score;
    this.dropChance = dropChance;
}
```

Constructor นี้เป็น `protected` เพราะ `Enemy` เป็น abstract class จึงถูกเรียกได้จากคลาสลูกเท่านั้น `this` ใช้แยกแอตทริบิวต์ของวัตถุออกจากพารามิเตอร์ที่ชื่อเหมือนกัน

ตัวอย่างที่ 2: คลาสลูกเรียกคลาสแม่ด้วย `super`

```
public Drone(double x, double y, Pattern pattern) {
    super(x, y, 44, 26, 3, 100, 0.18);
    this.pattern = pattern;
    this.baseY = y;
    this.hoverScreenX = 760 + Math.random() * 130; // อยู่หน้าปากกระบอกปืนผู้เล่นเสมอ
    this.fireTimer = 1.0 + Math.random() * 1.5;
}
```

`Drone` ส่งขนาด เลือด คะแนน และโอกาสดรอปของให้ `Enemy` ด้วย `super(...)` ก่อน แล้วจึงกำหนดค่าเฉพาะของตัวเอง

ตัวอย่างที่ 3: Constructor ที่สร้างและประกอบวัตถุอื่น

```
public AssaultGame(Input input, RunState run, AssaultHost host) {
    this.input = input;
    this.host = host;
    this.startRun = run.copy();
    this.stage = Stage.create(run.stage);
    this.level = new Level(stage);
    this.bg = new ParallaxBackground(stage);
    this.player = new AssaultPlayer(80, centerY() - 18);
    player.lives = run.lives;
    player.bombs = run.bombs;
    this.score = run.score;
    this.slug = new NovaSlug(stage.vehicleX(), centerY());
```

เมื่อเริ่มด่าน Constructor จะสร้างด่าน ฉาก ผู้เล่น และรถถังให้ครบ และรับค่าชีวิต ระเบิด และคะแนนจาก `RunState` เพื่อให้เล่นต่อจากด่านก่อนหน้าได้

2.4.2 Encapsulation

Encapsulation คือการซ่อนข้อมูลไว้ในคลาสและให้ภายนอกใช้ผ่านเมธอด เพื่อให้คลาสควบคุมความถูกต้องของข้อมูลเองได้

ตัวอย่างที่ 1: `Inventory` ซ่อนข้อมูลไว้ทั้งหมด

```
public class Inventory {
    private int hearts;
    private final Deque<ItemType> queue = new ArrayDeque<>();

    public int hearts() { return hearts; }

    /** ใช้หัวใจ 1 ดวง (คืน true ถ้ามีให้ใช้) */
    public boolean consumeHeart() {
        if (hearts <= 0) return false;
        hearts--;
        return true;
    }
```

ภายนอกแก้ `hearts` โดยตรงไม่ได้ จึงไม่มีทางทำให้จำนวนหัวใจติดลบ เมธอด `queued()` ยังคืนสำเนาของรายการแทนคิวจริง คลาสอื่นจึงแก้คิวภายในไม่ได้

ระดับการเข้าถึงที่ใช้ในเกม

ระดับ	ตัวอย่าง	เหตุผล
`private`	`AssaultGame.enemies`, `Inventory.hearts`	ข้อมูลภายในที่ต้องแก้ผ่านเมธอด
`protected`	`Enemy.t`, `Enemy.flash`, `Enemy.update()`	ให้คลาสลูกใช้ได้ แต่คลาสอื่นใช้ไม่ได้
`public`	`AssaultGame.spawn()`, `Enemy.damage()`	เมธอดที่เปิดให้คลาสอื่นเรียก

ข้อจำกัดที่ยอมรับไว้: แอตทริบิวต์ที่อ่านบ่อยทุกเฟรม เช่น `Enemy.x`, `Enemy.y` และ `Bullet.vx` เป็น `public` เพื่อให้โค้ดเกมสั้นลง คลาสอื่นจึงแก้ค่าเหล่านี้ได้โดยตรง ซึ่งเป็นจุดที่ปรับให้เข้มงวดขึ้นได้ในอนาคต

2.4.3 Composition

Composition คือการประกอบวัตถุจากวัตถุอื่น (ความสัมพันธ์แบบ has-a) โดยวัตถุส่วนประกอบถูกสร้างและถูกดูแลโดยวัตถุเจ้าของ

ตัวอย่างที่ 1: `GamePanel` ประกอบจากวัตถุของโหมด CLASSIC

```
private final Input input = new Input();
private final PlayerShip player = new PlayerShip();
private final AlienShip alien = new AlienShip();
private final List<Laser> lasers = new ArrayList<>();
private final List<Particle> particles = new ArrayList<>();
private final Background background = new Background();
private final Inventory inventory = new Inventory();
```

วัตถุเหล่านี้ถูกสร้างพร้อม `GamePanel` และประกาศเป็น `final` จึงอยู่กับ `GamePanel` ตลอดอายุการใช้งาน

ตัวอย่างที่ 2: ใช้ซ้ำโดยการประกอบแทนการสืบทอด

```
private final PlayerShip look = new PlayerShip();
```

`AssaultPlayer` ใช้ `PlayerShip` เพื่อวาดรูปยานเท่านั้น เพราะกติกา HP ปืน และระเบิดแตกต่างจากยานในโหมด CLASSIC การประกอบจึงเหมาะกว่าการสืบทอด

ตัวอย่างที่ 3: Aggregation เปรียบเทียบกับ Composition

```
public NovaSlug vehicle;          // != null = กำลังขับรถถัง
```

`AssaultPlayer` เพียงอ้างถึงรถถังขณะขับ แต่รถถังถูกสร้างโดย `AssaultGame` และคงอยู่ก่อนผู้เล่นขึ้นรถ จึงเป็น Aggregation และแสดงด้วยเพชรโปร่งในรูปที่ 2-2

2.4.4 Polymorphism

Polymorphism คือการเรียกเมธอดชื่อเดียวกันแล้วแต่ละวัตถุทำงานต่างกันตามชนิดจริงของวัตถุ เกมนี้ใช้ทั้งแบบ Overriding ซึ่งเลือกเมธอดตอนรันโปรแกรม และแบบ Overloading ซึ่งเลือกเมธอดตอนคอมไพล์

ตัวอย่างที่ 1: ลิสต์เดียวเก็บศัตรูได้ทุกชนิด

```
private final List<Enemy> enemies = new ArrayList<>();

for (Enemy e : enemies) e.tick(dt, this);
```

ตัวแปร `e` มีชนิดเป็น `Enemy` แต่วัตถุจริงอาจเป็น `Drone`, `Turret` หรือ `HiveQueenBoss` เมื่อ `tick()` เรียก `update()` โปรแกรมจะทำงานตาม `update()` ของชนิดจริง `AssaultGame` จึงไม่ต้องใช้ `if` แยกชนิดศัตรู

ตัวอย่างที่ 2: ศัตรูแต่ละชนิด Override พฤติกรรมของตัวเอง

```
// ค่าเริ่มต้นใน Enemy: กระสุนเข้าทุกนัด
public boolean blocksBullet(Bullet b) { return false; }

// ShieldDrone: โล่กันกระสุนที่ยิงมาตรง ๆ
@Override
public boolean blocksBullet(Bullet b) {
    if (shield <= 0 || b.pierce || b.explosive) return false;
    // ยิงมาจากด้านหน้าแบบเกือบแนวนอน = โดนโล่
    return b.vx > 0 && Math.abs(b.vy) < Math.abs(b.vx) * 0.2;
}
```

ตอนตรวจการชน `AssaultGame` เรียก `e.blocksBullet(b)` กับศัตรูทุกตัวเหมือนกัน แต่ได้ผลต่างกันตามชนิดของศัตรู

ตัวอย่างที่ 3: Overloading เมธอดชื่อเดียวกันแต่พารามิเตอร์ต่างกัน

```
/** โดนโจมตี 1 ครั้ง */
public void hit(AssaultGame g) {
    hit(g, Balance.NORMAL_HIT);
}

/** โดนโจมตี โดยระบุดาเมจ (ท่าใหญ่ของบอส = Balance.BIG_HIT) */
public void hit(AssaultGame g, int damage) {
```

การโจมตีทั่วไปเรียก `hit(g)` ส่วนท่าใหญ่ของบอสเรียก `hit(g, damage)` เพื่อระบุดาเมจเอง

2.4.5 Abstract

Abstract class ใช้กำหนดสิ่งที่คลาสลูกทุกตัวต้องมี แต่ปล่อยรายละเอียดให้คลาสลูกเขียนเอง สร้างวัตถุจาก abstract class โดยตรงไม่ได้ เกมนี้มี abstract class 3 คลาส และ interface 1 ตัว

ชนิด	ชื่อ	สิ่งที่บังคับให้คลาสลูกเขียน
abstract class	`Enemy`	`update()`, `draw()`
abstract class	`BossEnemy`	`displayName()`, `phase()`
abstract class	`Stage`	`number()`, `name()`, `length()`, `theme()`, `build()`, `createBoss()` และเมธอดข้อมูลด่านอื่น ๆ รวม 12 เมธอด
interface	`AssaultHost`	`toMenu()`, `toStageSelect()`, `startStage()`

ตัวอย่างที่ 1: `Enemy` กำหนดขั้นตอนหลักไว้ แล้วให้คลาสลูกเติมรายละเอียด

```
public abstract class Enemy {

    public final void tick(double dt, AssaultGame g) {
        if (dead || removed) return;
        if (delay > 0) { delay -= dt; return; }
        t += dt;
        if (flash > 0) flash -= dt;
        update(dt, g);
        if (!isBoss() && x < g.camX() - 160) removed = true;
    }

    protected abstract void update(double dt, AssaultGame g);

    protected abstract void draw(Graphics2D g, double time);
```

`tick()` เป็น `final` ทำให้ศัตรูทุกตัวผ่านขั้นตอนเดียวกัน ส่วน `update()` เป็น `abstract` เพราะศัตรูแต่ละชนิดเคลื่อนที่ไม่เหมือนกัน แนวทางนี้ตรงกับรูปแบบ Template Method

ตัวอย่างที่ 2: `Stage` บังคับให้ทุกด่านมีข้อมูลครบ

```
public abstract class Stage {
    public abstract int number();
    public abstract String name();
    public abstract double length();           // ความยาวแมพทั้งหมด

    /** สร้างศัตรู/เหตุการณ์ทั้งหมดของด่าน */
    public abstract void build(Level level);

    /** สร้างบอสของด่าน */
    public abstract BossEnemy createBoss(AssaultGame g);

    /** อัปเดตกลไกพิเศษทุกเฟรม (เช่น พายุหิมะ) */
    public void update(double dt, AssaultGame g) {}
```

เมธอดที่ทุกด่านต้องมีเป็น `abstract` ส่วนกลไกที่บางด่านไม่มี เช่น `update()` เป็นเมธอดปกติที่ไม่ทำอะไร คลาสลูกจึงเลือก Override เฉพาะที่ต้องใช้

ตัวอย่างที่ 3: Interface

```
public interface AssaultHost {
    void toMenu();

    void toStageSelect();

    void startStage(RunState run);
}
```

`AssaultHost` กำหนดเฉพาะสิ่งที่โหมด ASSAULT ขอให้หน้าจอหลักทำ โดยไม่มีข้อมูลหรือการทำงานอยู่ภายใน

2.4.6 Inheritance

Inheritance คือการสร้างคลาสใหม่โดยรับคุณสมบัติและเมธอดจากคลาสแม่ (ความสัมพันธ์แบบ is-a) ดูภาพรวมได้จากรูปที่ 2-3

คลาสแม่	คลาสลูก	สิ่งที่ได้รับจากคลาสแม่
`JPanel`	`GamePanel`	ความสามารถของ Swing panel เช่น `repaint()`, `addKeyListener()`
`KeyAdapter`	`Input`	เขียนเฉพาะ `keyPressed()` และ `keyReleased()` โดยไม่ต้องเขียน `keyTyped()`
`Enemy`	ศัตรู 16 คลาส, `QueenEye` และ `BossEnemy`	ตำแหน่ง เลือด กล่องชน การรับดาเมจ และการตาย
`BossEnemy`	`MothershipBoss`, `CryoSerpentBoss`, `HiveQueenBoss`	ทุกอย่างจาก `Enemy` รวมกับข้อมูลสำหรับแถบเลือดบอส
`Stage`	`Stage1NovaSector`, `Stage2FrozenRings`, `Stage3HiveCore`	เมธอดกล้องและค่าเริ่มต้นของกลไกพิเศษ

ตัวอย่างที่ 1: การสืบทอดหลายชั้น

```
public abstract class Enemy {

public abstract class BossEnemy extends Enemy {

    protected BossEnemy(double x, double y, double w, double h, double hp, int score) {
        super(x, y, w, h, hp, score, 0);
    }

    @Override
    public boolean isBoss() { return true; }

public class HiveQueenBoss extends BossEnemy {
```

`HiveQueenBoss` สืบทอดจาก `BossEnemy` และ `BossEnemy` สืบทอดจาก `Enemy` บอสจึงส่งค่าโอกาสดรอปของเป็น 0 ให้คลาสแม่ และเปลี่ยน `isBoss()` เป็น `true`

ตัวอย่างที่ 2: การ Override และเรียกเมธอดเดิมด้วย `super`

```
@Override
public void damage(double d, AssaultGame g) {
    if (!exposed()) return;
    super.damage(d, g);
}
```

ราชินีรับดาเมจไม่ได้จนกว่าตาทั้ง 3 ดวงจะแตก เมื่อเปิดหัวใจแล้วจึงเรียก `super.damage()` เพื่อใช้วิธีคำนวณดาเมจเดิมของ `Enemy` โดยไม่ต้องเขียนซ้ำ

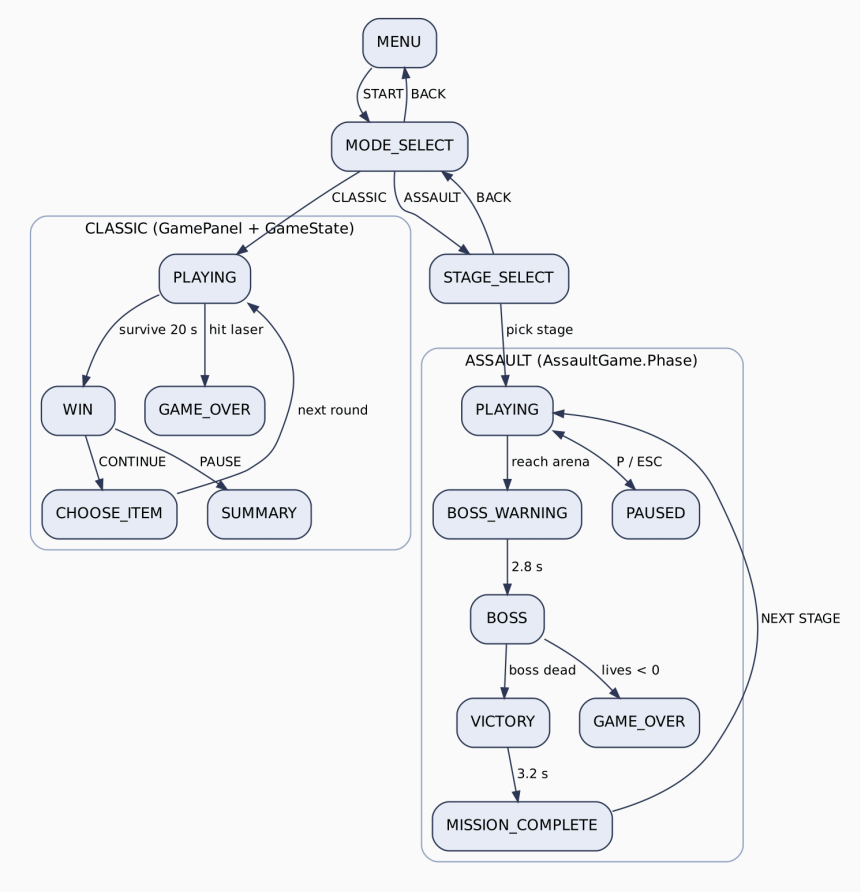
2.5 หน้าจอ GUI

2.5.1 โครงสร้างของ GUI

GUI ใช้ Swing Component จริงเพียง 2 ตัว คือ `JFrame` และ `GamePanel` ส่วนปุ่ม การ์ด และแถบข้อมูลทุกอย่างวาดด้วย `Graphics2D` ในเมธอด `paintComponent` แนวทางนี้ทำให้ควบคุมหน้าตาและแอนิเมชันได้เต็มที่ และไม่ต้องใช้ไฟล์ภาพ

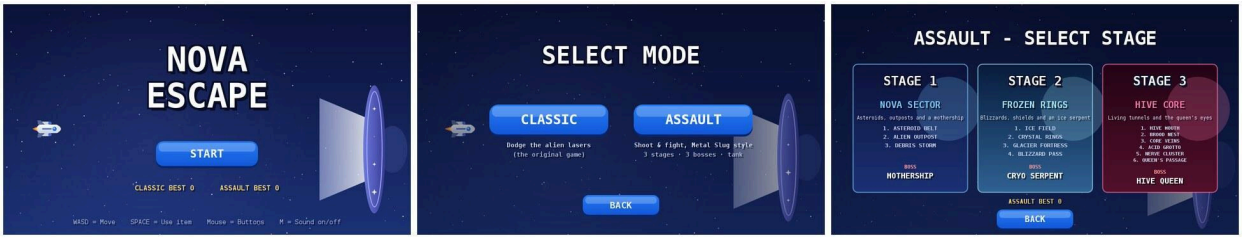
Component	ชนิด	หน้าที่
`JFrame`	Swing Container	หน้าต่างโปรแกรม ปรับขนาดไม่ได้
`GamePanel`	สืบทอดจาก `JPanel`	พื้นที่วาดทุกหน้าจอ ขนาด 960 × 540 พิกเซล
`Timer`	`javax.swing.Timer`	เรียก game loop ทุก 16 มิลลิวินาที ไม่แสดงผลบนจอ
`Button`	คลาสที่เขียนเอง	ปุ่มไล่สี มีเงา และยกตัวเมื่อเมาส์ชี้
การ์ดด่าน / การ์ดไอเทม	วาดด้วย `Graphics2D`	พื้นที่คลิกตาม `Rectangle` ที่คำนวณไว้
HUD	วาดด้วย `Graphics2D`	แสดง HP ชีวิต ปืน ระเบิด ไอเทม ความคืบหน้า และคะแนน

แต่ละเฟรมวาดใหม่ทั้งจอจากชั้นหลังไปชั้นหน้า สถานะ `GameState` เป็นตัวเลือกว่าต้องวาดหน้าใด และ `setDoubleBuffered(true)` ช่วยลดภาพกระพริบ



รูปที่ 2-4 การเปลี่ยนหน้าจอและสถานะของเกม

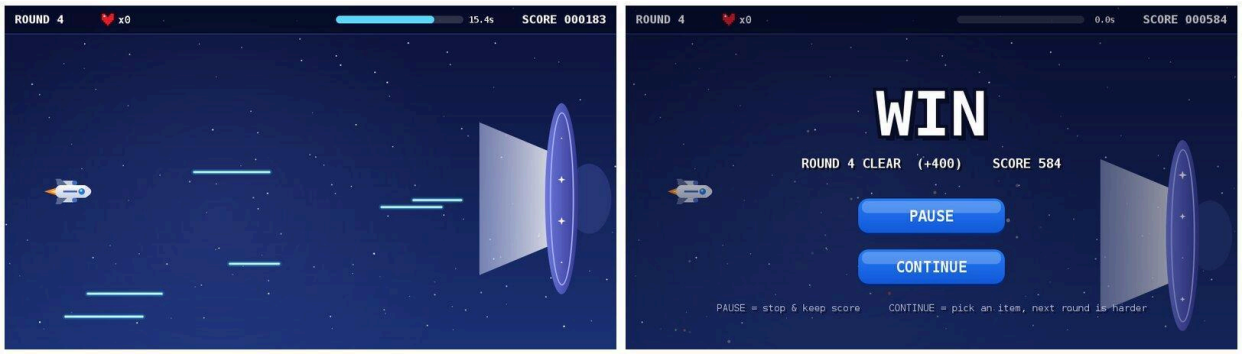
2.5.2 หน้าจอเมนู



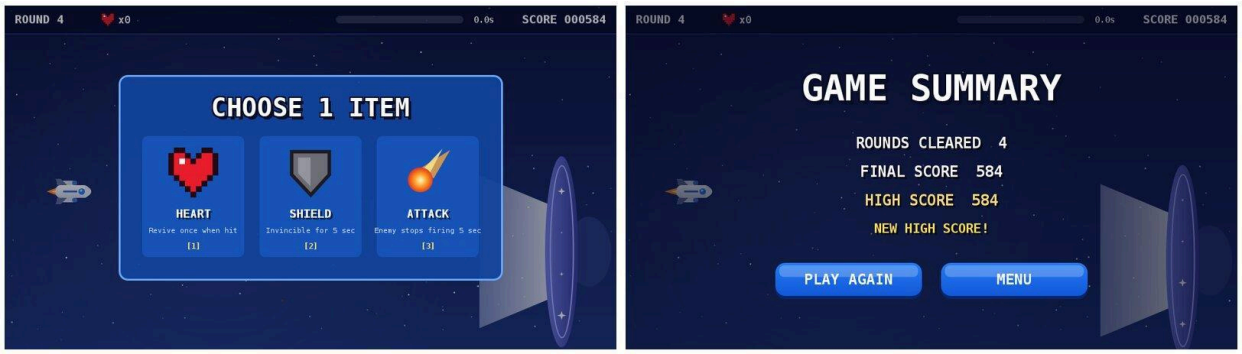
รูปที่ 2-5 จากซ้ายไปขวา: หน้าเริ่มเกม (MENU), หน้าเลือกโหมด (MODE_SELECT) และหน้าเลือกด่าน (STAGE_SELECT)

หน้าจอ	ส่วนประกอบ
MENU	ฉากดาว ยานผู้เล่น ยานต่างดาว ชื่อเกม ปุ่ม START คะแนนสูงสุดของทั้ง 2 โหมด และคำแนะนำปุ่มควบคุม
MODE_SELECT	ปุ่ม CLASSIC, ASSAULT และ BACK พร้อมคำอธิบายของแต่ละโหมด
STAGE_SELECT	การ์ด 3 ใบแสดงชื่อด่าน เขตย่อย และบอส โดยดึงข้อมูลจากคลาส `Stage` มีคะแนนสูงสุดและปุ่ม BACK

2.5.3 หน้าจอโหมด CLASSIC



รูปที่ 2-6 ซ้าย: หน้าเล่น (PLAYING) ขวา: หน้าชนะ (WIN)



รูปที่ 2-7 ซ้าย: หน้าเลือกไอเทม (CHOOSE_ITEM) ขวา: หน้าสรุปคะแนน (SUMMARY)

หน้าจอ	ส่วนประกอบ
PLAYING	แถบด้านบนแสดงเลขตา จำนวนหัวใจ ไอเทมในคิว แถบเวลาถอยหลัง และคะแนน พื้นที่เล่นมียานผู้เล่น เลเซอร์ และยานต่างดาว
WIN	ข้อความ WIN โบนัสผ่านตา คะแนน ปุ่ม PAUSE และ CONTINUE
CHOOSE_ITEM	การ์ด 3 ใบแสดงไอคอน ชื่อ และคำอธิบายจาก `ItemType` เลือกได้ด้วยเมาส์หรือปุ่ม 1 2 3
SUMMARY	จำนวนตาที่ผ่าน คะแนนสุดท้าย คะแนนสูงสุด ปุ่ม PLAY AGAIN และ MENU
GAME_OVER	ข้อความ GAME OVER คะแนน เลขตา ปุ่ม PLAY AGAIN และ MENU

2.5.4 หน้าจอโหมด ASSAULT



รูปที่ 2-8 ซ้าย: ด่าน 1 Nova Sector และรถถังที่จอดรอ ขวา: ขณะขับรถถัง Nova Slug

HUD จากซ้ายไปขวาแสดง HP และชีวิตสำรอง ชื่อปืนและกระสุน จำนวนระเบิด ไอเทมที่กด Space ใช้ได้ แถบความคืบหน้าของด่านและคะแนน ขณะขับรถถัง ช่องชื่อปืนจะเปลี่ยนเป็นแถบเกราะและเวลาที่เหลือ และมีวงกลมสีเหลืองเป็นเป้าเล็งตามเมาส์



รูปที่ 2-9 ซ้าย: ห้องบอส Mothership ขวา: หน้าหยุดชั่วคราว (PAUSED)

ในห้องบอส กล้องซูมออกให้เห็นกว้างขึ้น และแถบความคืบหน้าเปลี่ยนเป็นแถบเลือดและเฟสของบอส หน้า PAUSED มีปุ่ม RESUME, STAGE SELECT และ MENU



รูปที่ 2-10 ซ้าย: ด่าน 2 Frozen Rings พร้อมพายุหิมะ ขวา: บอส Cryo Serpent



รูปที่ 2-11 ซ้าย: ด่าน 3 Hive Core พร้อมผนังอุโมงค์ ขวา: บอส Hive Queen และตา 3 ดวง



รูปที่ 2-12 ซ้าย: หน้าจบด่าน (MISSION_COMPLETE) ขวา: หน้าแพ้ (GAME_OVER)

หน้าจบด่านแสดงจำนวนศัตรู โบนัส คะแนนรวม และปุ่มไปต่อ ส่วนหน้าแพ้มีปุ่ม RETRY, STAGE SELECT และ MENU

2.6 Event Handling

โปรแกรมรับ Event 3 ชนิด คือ คีย์บอร์ด เมาส์ และ Timer โดยทุก Event ทำงานบน Event Dispatch Thread ตามรูปแบบของ Swing

Event	Listener / Adapter	ตำแหน่ง	หน้าที่
`KeyEvent`	`KeyAdapter`	`Input`	เก็บสถานะปุ่มทุกหน้าจอ
`MouseEvent` (กด / ปล่อย)	`MouseAdapter`	คลาสไม่ระบุชื่อใน `GamePanel`	กดปุ่ม ยิง และใช้ระเบิด
`MouseEvent` (ขยับ / ลาก)	`MouseAdapter`	คลาสไม่ระบุชื่อใน `GamePanel`	แสดง hover และเล็งป้อมปืนรถถัง
`ActionEvent`	Lambda	คอนสตรัคเตอร์ `GamePanel`	เรียก `tick()` ทุก 16 มิลลิวินาที

ตัวอย่างที่ 1: การรับคีย์บอร์ด

```
@Override
public void keyPressed(KeyEvent e) {
    set(e.getKeyCode(), true);
    int code = e.getKeyCode();
    if (code == KeyEvent.VK_SPACE) spaceQueued = true;
    if (code == KeyEvent.VK_K) bombQueued = true;
    if (code == KeyEvent.VK_P || code == KeyEvent.VK_ESCAPE) pauseQueued = true;
    if (code == KeyEvent.VK_M) novaescape.audio.Sound.toggleMute();   // M = เปิด/ปิดเสียง
}

@Override
public void keyReleased(KeyEvent e) {
    set(e.getKeyCode(), false);
}
```

ปุ่มที่กดค้าง เช่น W A S D จะเก็บเป็น `boolean` ทำให้กดหลายปุ่มพร้อมกันและเคลื่อนที่แนวทแยงได้ ส่วนปุ่มที่กดครั้งเดียว เช่น Space และ K จะเข้าคิวไว้ให้ game loop อ่าน

ตัวอย่างที่ 2: การรับเมาส์

```
MouseAdapter mouseHandler = new MouseAdapter() {
    @Override public void mouseMoved(MouseEvent e) { onMouseMove(e.getPoint()); }
    @Override public void mouseDragged(MouseEvent e) { onMouseMove(e.getPoint()); }
    @Override public void mousePressed(MouseEvent e) {
        requestFocusInWindow();
        if (state == GameState.ASSAULT && assault != null) {
            // คลิกโดนปุ่ม = กดปุ่ม, ไม่โดน = ซ้ายยิง / ขวาระเบิด
            if (assault.mouseClick(e.getPoint())) return;
            if (assault.isInMenuScreen()) return;
            if (SwingUtilities.isLeftMouseButton(e)) input.mouseShoot = true;
            if (SwingUtilities.isRightMouseButton(e)) input.queueBomb();
            return;
        }
        onMouseClick(e.getPoint());
    }
    @Override public void mouseReleased(MouseEvent e) {
        if (SwingUtilities.isLeftMouseButton(e)) input.mouseShoot = false;
    }
};
addMouseListener(mouseHandler);
addMouseMotionListener(mouseHandler);
```

คลาสไม่ระบุชื่อตัวเดียวถูกลงทะเบียนทั้งการคลิกและการขยับเมาส์ ระหว่างเล่นโหมด ASSAULT การคลิกจะตรวจปุ่มก่อน ถ้าไม่โดนปุ่มจึงนับเป็นการยิง

ตัวอย่างที่ 3: ปุ่มที่เขียนเองรับคำสั่งเป็น `Runnable`

```
// สร้างปุ่ม พร้อมระบุสิ่งที่ต้องทำเมื่อถูกกด
buttons.add(new Button("START", cx, 350, 240, 58, this::goToModeSelect));

/** คืน true ถ้าคลิกโดนปุ่มนี้ */
public boolean click(Point p) {
    if (bounds.contains(p)) {
        novaescape.audio.Sound.play(novaescape.audio.Sfx.CLICK);
        onClick.run();
        return true;
    }
    return false;
}
```

`GamePanel` วนตรวจทุกปุ่มว่าตำแหน่งคลิกอยู่ใน `bounds` หรือไม่ ถ้าใช่จะเรียกคำสั่งที่ผูกไว้ การส่ง method reference ทำให้แต่ละปุ่มมีหน้าที่ต่างกันได้โดยไม่ต้องเขียน `if` แยกชื่อปุ่ม

ตัวอย่างที่ 4: Timer ขับ game loop

```
loop = new Timer(Config.FRAME_MS, e -> tick());
```

`Timer` ส่ง `ActionEvent` ทุก 16 มิลลิวินาที คิดเป็นเป้าหมายประมาณ 60 เฟรมต่อวินาที

2.7 อัลกอริทึมที่สำคัญในโปรแกรม

2.7.1 Game Loop แบบ Delta Time

```
private void tick() {
    long now = System.nanoTime();
    double dt = Math.min(0.05, (now - lastNanos) / 1_000_000_000.0);
    lastNanos = now;
    update(dt);
    repaint();
}
```

1. หาเวลาที่ผ่านไปจากเฟรมก่อนหน้าเป็นวินาที (`dt`)
2. จำกัด `dt` ไม่เกิน 0.05 วินาที เพื่อไม่ให้วัตถุกระโดดไกลเมื่อเครื่องกระตุก
3. อัปเดตตำแหน่งทุกวัตถุด้วย `x += vx * dt`
4. เรียก `repaint()` เพื่อวาดหน้าจอใหม่

การคูณด้วย `dt` ทำให้ยานเคลื่อนที่ด้วยความเร็วเท่ากันในทุกเครื่อง แม้อัตราเฟรมจะต่างกัน

2.7.2 การตรวจจับการชนแบบ AABB

วัตถุแต่ละชิ้นมีกล่องชน `Rectangle2D` และตรวจการชนด้วย `intersects()` กล่องชนของยานผู้เล่นเล็กกว่ารูปยานเล็กน้อย เพื่อให้ผู้เล่นรู้สึกว่าเกมยุติธรรม

กระสุนผู้เล่นมีลำดับการตรวจดังนี้

```
for (Enemy e : enemies) {
    if (!e.active() || b.alreadyHit.contains(e)) continue;
    if (!bb.intersects(e.hitbox())) continue;
    if (e.blocksBullet(b)) {
        e.onBlocked(b, this);
        b.dead = true;
        sparks(b.x, b.y, new Color(150, 220, 255), 4);
        return;
    }
    hitSomething = true;
    e.damage(b.damage, this);
    sparks(b.x, b.y, new Color(255, 230, 150), 3);
    if (b.explosive) splash(b, e);
    if (b.pierce) {
        b.alreadyHit.add(e);
    } else {
        b.dead = true;
        return;
    }
}
```

1. ข้ามศัตรูที่ไม่อยู่ในสถานะใช้งาน และศัตรูที่กระสุนทะลุไปแล้ว
2. ถ้าโดนโล่ กระสุนจะหายไปและลดพลังโล่แทน
3. ถ้าโดนตัวศัตรู จะลดเลือด และกระสุนจรวดจะทำดาเมจรอบข้าง
4. กระสุนเลเซอร์ทะลุต่อได้ โดยจำศัตรูไว้ใน `alreadyHit` เพื่อไม่ให้โดนตัวเดิมซ้ำ

2.7.3 การเพิ่มความยากตามเลขตา (โหมด CLASSIC)

ยิ่งตาสูง เลเซอร์ยิ่งเร็วและยิงถี่ขึ้น ตามสูตรในคลาส `Difficulty` ตารางนี้แสดงค่าที่คำนวณได้ในบางตา

ตา	ความเร็วเลเซอร์ (พิกเซล/วินาที)	ระยะห่างการยิง (วินาที)	เลเซอร์ต่อชุด	โอกาสเล็งผู้เล่น
1	340	0.85	1	0%
3	450	0.63	2	24%
5	560	0.46	3	48%
8	725	0.30	4	55%
10	835	0.22	5	55%

`Math.max` และ `Math.min` กำหนดเพดานความยาก ทำให้ตาสูงยังมีโอกาสหลบได้

2.7.4 สคริปต์ด่านและการล็อกกล้อง (โหมด ASSAULT)

แต่ละด่านเขียนเหตุการณ์ไว้ตามตำแหน่งกล้อง เมื่อกล้องถึงตำแหน่งที่กำหนด เหตุการณ์จะทำงาน 1 ครั้ง

```
// ตัวอย่างการเขียนด่าน
L.lockAt(1900, g -> {
    L.droneLine(g, 6, Drone.Pattern.HOVER, 0.5);
    L.kamikazes(g, 3, 1.2).forEach(e -> e.delay += 2.0);
});

// เรียกทุกเฟรม: เช็กว่ากล้องถึงเหตุการณ์ไหนแล้ว
public void update(AssaultGame g) {
    for (Event e : events) {
        if (e.done || g.camX() < e.at) continue;
        e.done = true;
        if (e.lock) g.beginLock(e.at);
        e.action.accept(g);
        if (e.lock) g.endLockRecording();
    }
}
```

`lockAt` จะล็อกกล้องแบบ Metal Slug จนกว่าศัตรูในระลอกนั้นจะหมด กล้องจึงเลื่อนได้ไม่เกิน `lockX`

กล้องเดินหน้าอย่างเดียวและเคลื่อนได้ไม่เกิน 430 พิกเซลต่อวินาที ถ้าล็อกนานเกิน 45 วินาที ศัตรูทั่วไปที่เหลือจะถูกนำออก เพื่อไม่ให้เกมติดค้าง

2.7.5 จรวดติดตามเป้าหมาย

```
private void steer(double dt, AssaultGame g) {
    Enemy target = null;
    double best = Double.MAX_VALUE;
    for (Enemy e : g.enemies()) {
        if (!e.targetable() || e.x < x - 40) continue;
        double d = Math.hypot(e.x - x, e.y - y);
        if (d < best) { best = d; target = e; }
    }
    double speed = Math.min(760, Math.hypot(vx, vy) + 900 * dt);
    double ang = Math.atan2(vy, vx);
    if (target != null) {
        double want = Math.atan2(target.y - y, target.x - x);
        double diff = Math.atan2(Math.sin(want - ang), Math.cos(want - ang));
        double maxTurn = 5.0 * dt;
        ang += Math.max(-maxTurn, Math.min(maxTurn, diff));
    }
```

1. หาศัตรูที่ใกล้ที่สุดซึ่งอยู่ด้านหน้าจรวด (Linear Search)
2. คำนวณมุมที่ต้องหันไปด้วย `atan2`
3. ปรับผลต่างของมุมให้อยู่ในช่วง -**π** ถึง **π** เพื่อให้เลี้ยวด้านที่ใกล้กว่า
4. จำกัดการเลี้ยวไม่เกิน 5 เรเดียนต่อวินาที จรวดจึงโค้งเข้าหาเป้าอย่างเป็นธรรมชาติ

บทที่ 3

สรุป

โครงงานนี้ทำเกม Nova Escape ได้ครบตาม Proposal และขยายเพิ่มเป็นโหมด ASSAULT 3 ด่าน โดยใช้หลัก OOP ครบทั้ง 6 หัวข้อที่กำหนด

3.1 ปัญหาที่พบระหว่างการพัฒนา

ปัญหา	สาเหตุ	วิธีแก้
โปรแกรมอาจเกิด `ConcurrentModificationException`	ศัตรูสร้างศัตรูหรือกระสุนใหม่ ขณะกำลังวนลิสต์เดิม	พักของใหม่ไว้ใน `spawnQueue` และ `newBullets` แล้วค่อยเพิ่มหลังวนเสร็จ
ความเร็วเกมไม่เท่ากันในแต่ละเครื่อง	อัตราเฟรมของแต่ละเครื่องต่างกัน	คำนวณการเคลื่อนที่ด้วย delta time และจำกัด `dt` ไม่เกิน 0.05 วินาที
เกมอาจติดค้างตอนล็อกกล้อง	ศัตรูบางตัวอยู่ในตำแหน่งที่ฆ่าได้ยาก	เพิ่ม `LOCK_TIMEOUT` 45 วินาที แล้วนำศัตรูทั่วไปที่เหลือออก
เกมเปิดช้าและอาจพังในเครื่องที่ไม่มีการ์ดเสียง	การสร้างเสียงใช้เวลา และอาจเปิดอุปกรณ์เสียงไม่ได้	แยกเธรดเสียง ดักข้อผิดพลาด และให้เกมเล่นต่อแบบไม่มีเสียง
ปรับสมดุลเกมยาก	ค่าความยากกระจายอยู่หลายคลาส	รวมค่าไว้ที่ `Config` และ `Balance`

ปัญหาที่ยังเหลืออยู่: `AssaultGame` มี 1,075 บรรทัด และ `GamePanel` มี 749 บรรทัด ทั้งสองคลาสรับหน้าที่หลายอย่าง ควรแยกส่วนการวาด HUD และการชนออกเป็นคลาสใหม่ในอนาคต

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

1. **มี 2 เกมในโปรแกรมเดียว** ทั้งเกมหลบกระสุนและเกมยิงแนว Metal Slug
2. **ไม่มีไฟล์ภาพและไฟล์เสียงเลย** ทุกฉากวาดด้วย `Graphics2D` ส่วนเสียงและเพลงแนว 8-bit สังเคราะห์ด้วยโค้ดทั้งหมด
3. **บอสเล่นต่างกัน** Mothership เน้นหลบกระสุน, Cryo Serpent ต้องเล็งหัวงู, Hive Queen ต้องทำลายตา 3 ดวงก่อน
4. **เพิ่มเนื้อหาได้ง่าย** ศัตรูใหม่เพียงสืบทอด `Enemy` และด่านใหม่เพียงสืบทอด Stage

3.3 คำแนะนำสำหรับผู้สอน

สิ่งที่อยากให้อธิบายเพิ่ม หรือเพิ่มในการเรียนสำหรับน้อง ๆ รุ่นต่อไป มีดังนี้

1. **Game Loop และ Thread ใน Swing:** อยากให้มีตัวอย่างการทำแอนิเมชันด้วย Timer หรือ Thread
2. **Composition เทียบกับ Inheritance:** อยากให้มีตัวอย่างว่าเมื่อใดควรเลือกใช้แบบใด